

Gold Layer — Business Metrics & Aggregations

ServiceTrack Project | Execution Screenshots

Screenshot 1: Metric 1 - Technician Performance

FileEditViewRunHelpPython ☆Last edit was 21 minutes ago

Run allServerlessSchedule

Python

Just now (8s)7

```
1 # Compute technician performance statistics
2 df_tech_performance = df_silver.groupBy("technician_id", "technician_name") \
3     .agg(
4         count("job_id").alias("total_jobs"),
5         sum(when(col("job_status") == "Completed", 1).otherwise(0)).alias("completed_jobs"),
6         avg("Repair_Duration").alias("avg_repair_duration_days"),
7         sum(when(col("completed_date") > col("promised_date"), 1).otherwise(0)).alias("delayed_jobs")
8     ) \
9     .withColumn(
10         "delay_rate_pct",
11         # Fix: avoid divide-by-zero when a technician has 0 completed jobs (returns null instead)
12         when(col("completed_jobs") > 0, (col("delayed_jobs") / col("completed_jobs")) * 100).otherwise(None)
13     )
14
15 # Display the metrics
```

Table +

SearchFilterColumnsTerminal (+2)

	technician_id	technician_name	total_jobs	completed_jobs	avg_repair_duration_days	delayed_jobs	delay_rate_pct
1	T005	Kavitha Nair	192	143	2.569620253164557	0	0
2	T006	Deepak Sharma	186	135	5.246478873239437	63	46.666666666666664
3	T007	Meena Reddy	209	152	4.166666666666667	24	15.789473684210526
4	T008	Arjun Iyer	159	111	7.094827586206897	88	79.27927927927928

5 rows | 8.487s runtime

Refreshed no

Average repair duration and delay rate calculated per technician, with a divide-by-zero safeguard applied.

Screenshot 2: Metric 2 - Delay Analysis



The screenshot shows a Jupyter Notebook interface with a Python kernel. The code defines a function to calculate actual delay using 'completed_date' and 'promised_date', then aggregates metrics for all jobs and delayed jobs only. The resulting table shows the following data:

	1.2 avg_delay_days_all_jobs	1.2 avg_delay_days_delayed_only	1.2 overall_delay_rate_pct
1	0.4433333333333333	2.239057239057239	19.8

1 row | 1.339s runtime

Overall delay metrics calculated using actual promised_date, showing both average delay (all jobs) and average delay (delayed jobs only).

Screenshot 3: Metric 3 - Repeat Customers

Metric 3 - Repeat Customers

▶

✓ Just now (2s)

11

Python

🗑️

🔗

📄

⋮

```
1 # Count jobs grouped by customer and issue category
2 df_repeat_customers = df_silver.groupBy("customer_id", "customer_name", "issue_type") \
3     .agg(count("job_id").alias("visit_count")) \
4     .filter(col("visit_count") > 1) \
5     .sort(col("visit_count").desc())
6
7 display(df_repeat_customers.limit(5))
```

> [See performance \(1\)](#) ✓ Checked for improvements

> df_repeat_customers: pyspark.sql.connect.dataframe.DataFrame = [customer_id: string, customer_name: string ... 2 more fields]

Table ▾ +

🔍

🏠

⌵

📄

	^A ₀ customer_id	^A ₀ customer_name	^A ₀ issue_type	¹ ₃ visit_count
1	CUST0231	Neha Fernandez	Charging Port Fault	6
2	CUST0035	Gaurav Chaudhary	Wi-Fi Not Connecting	5
3	CUST0274	Vimala Qureshi	RAM Issue	4
4	CUST0045	Zara Mishra	Charging Port Fault	4
5	CUST0192	Sonam Iyer	Wi-Fi Not Connecting	4

⬇️

▾

5 rows | 1.674s runtime

Refreshed now

Customers returning multiple times for the same issue type, sorted by visit count.

Screenshot 4: Metric 4 - Device Brand Analysis

Metric 4 - Device Brand Analysis.

▶

✓ 1 minute ago (1s)

15

Python

🗑️

🔗

🔗

⋮

```
1 # Aggregate volume of repair jobs by brand and type
2 df_brand_analysis = df_silver.groupBy("brand", "device_type", "issue_type") \
3     .agg(
4         count("job_id").alias("total_jobs"),
5         avg("estimated_cost").alias("avg_estimated_cost"),
6         avg("actual_cost").alias("avg_actual_cost")
7     ) \
8     .sort(col("total_jobs").desc())
9
10 display(df_brand_analysis.limit(5))
```

> [See performance \(1\)](#)

✓ Checked for improvements

> df_brand_analysis: pyspark.sql.connect.dataframe.DataFrame = [brand: string, device_type: string ... 4 more fields]

Table

+

🔍

🔍

🔍

🔍

	brand	device_type	issue_type	total_jobs	avg_estimated_cost	avg_actual_cost
1	Motorola	Printer	Wi-Fi Not Connecting	9	3383.5611111111111	5054.2675
2	Sony	Monitor	Charging Port Fault	9	5131.7211111111112	4103.24875
3	Lenovo	Monitor	Wi-Fi Not Connecting	8	3906.43625	5122.4050000000001

⬇

5 rows | 1.389s runtime

Refreshed 1 minute ago

Repair job volume and average costs aggregated by device brand, type, and issue category.

Screenshot 5: Metric 5 - Customer Latest Visit

Metric 5 - Customer Latest Visit

Just now (1s) 17 Python

```
1 # Create window partition to rank visits per customer
2 window_spec = Window.partitionBy("customer_id").orderBy(col("received_date").desc(), col("job_id").desc())
3
4 # Apply window row_number ranking
5 df_customer_latest_visit = df_silver.withColumn("row_num", row_number().over(window_spec)) \
6   .filter(col("row_num") == 1) \
7   .select("customer_id", "customer_name", "job_id", "received_date", "job_status", "actual_cost")
8
9 display(df_customer_latest_visit.limit(5))
```

> df_customer_latest_visit: pyspark.sql.connect.dataframe.DataFrame = [customer_id: string, customer_name: string ... 4 more fields]

	customer_id	customer_name	job_id	received_date	job_status	actual_cost
1	CUST0001	Kiran Patel	JOB00991	2024-03-13	Cancelled	null
2	CUST0003	Bhavna Tiwari	JOB00107	2024-03-18	Completed	2193.57
3	CUST0004	Syed Rao	JOB00193	2024-03-16	Pending	null
4	CUST0005	Suresh Bhat	JOB00553	2024-02-20	Completed	6572.34
5	CUST0006	Deepika Iyer	JOB00036	2024-03-10	Cancelled	null

5 rows | 1.395s runtime Refreshed now

Most recent job details extracted for each customer using a window function ranking.

Screenshot 6: All Gold Delta Tables Saved



The screenshot shows a code editor interface with a dark theme. The top bar includes a menu (File, Edit, View, Run, Help), a Python dropdown, a star icon, and a timestamp 'Last edit was 5 minutes ago'. On the right, there are buttons for 'Run all', 'Serverless', and 'Schedule'. The code editor displays Python code for saving three Delta tables: 'gold_technician_performance', 'gold_delay_analysis', and 'gold_repeat_customers'. The code uses the 'write.format("delta")' method and sets 'mode("overwrite")' and 'option("overwriteSchema", "true")'. The output pane at the bottom shows the message 'All 6 Gold Delta Tables saved successfully.'

```
1 # Save Technician Performance
2 df_tech_performance.write.format("delta") \
3     .mode("overwrite") \
4     .option("overwriteSchema", "true") \
5     .saveAsTable("gold_technician_performance")
6
7 # Save Delay Analysis
8 df_delay_analysis.write.format("delta") \
9     .mode("overwrite") \
10    .option("overwriteSchema", "true") \
11    .saveAsTable("gold_delay_analysis")
12
13 # Save Repeat Customers
14 df_repeat_customers.write.format("delta") \
15    .mode("overwrite") \
```

Output Terminal Debugger pip logs

All 6 Gold Delta Tables saved successfully.

All 6 Gold Delta tables (technician performance, delay analysis, repeat customers, repeat customers overall, device brand analysis, and customer latest visit) saved successfully.